

DeepXube Domain Design: Warehouse Robot Coordination (MAPF)

J.C. Vaught

April 2026

1 State Representation

The state encodes the positions of all K robots on an $H \times W$ grid warehouse floor, plus a static obstacle map. We fix $K = 4$ robots on an 8×8 grid for the initial implementation, scaling later to $K = 6$ on 12×12 .

The state is a single `int8` array of shape (H, W) where each cell takes one of the following values.

- 0: empty floor
- -1: static obstacle (shelf, wall, pillar)
- $1 \dots K$: robot ID occupying that cell

This gives a compact 2D representation that is easy to convolve over. For the 8×8 , $K = 4$ case the array is 64 bytes. The framework’s `State` object wraps this as a NumPy array of dtype `np.int8`.

An equivalent flat representation is also useful internally: a 1D array of shape $(K,)$ storing the flattened index ($\text{row} \times W + \text{col}$) for each robot, paired with a frozen obstacle mask of shape (H, W) stored once per environment instance (not per state). This dual view keeps state copying cheap (copy only the K -length position vector) while the grid view feeds the neural network.

Static obstacles are fixed at environment construction time via a binary mask. A reasonable density is 15–20% of cells, arranged in axis-aligned shelf rows with aisle gaps, mimicking a real warehouse layout.

2 Goal Representation

The goal is a target position assignment: robot i must reach cell (r_i, g_i) . The goal array mirrors the state array in shape (H, W) with the same encoding convention. Robot IDs $1 \dots K$ mark target cells; all other cells are 0 (obstacles are omitted from the goal array since they are environment-level constants, not per-instance data). The heuristic network receives the concatenation of state and goal arrays along a channel dimension, yielding input shape $(2, H, W)$.

3 Actions

All K robots move simultaneously at each timestep. A joint action is a tuple of K individual directions, one per robot, where each direction is drawn from $\{\text{UP}, \text{DOWN}, \text{LEFT}, \text{RIGHT}, \text{WAIT}\}$. The full action space is 5^K joint actions. For $K = 4$ this is 625; for $K = 6$ it is 15,625.

The domain uses `ActsEnum` (state-dependent actions). A joint action is valid only if no conflicts occur.

- *Vertex conflict*: two robots occupy the same cell after the move.
- *Edge conflict*: two robots swap positions (cross paths on the same edge).
- *Boundary/obstacle*: a robot’s target cell is out of bounds or an obstacle.

If any robot’s individual move is invalid (boundary/obstacle), that robot stays in place. If vertex or edge conflicts arise between robots, the entire joint action is rejected. `get_state_actions` enumerates valid joint actions by generating the Cartesian product of per-robot valid moves and filtering conflicts. For typical grid states with 4 robots, 50–200 joint actions survive out of 625.

3.1 Objective: Makespan

The optimization objective is *makespan*: the total number of simultaneous timesteps until all robots reach their destinations. Each transition has a uniform cost of 1.0. A robot that arrives at its goal early simply issues WAIT actions until all others arrive. This is the standard MAPF objective and maps directly to DeepXube’s cost-to-go framework, where $h(\text{state}, \text{goal})$ estimates the number of remaining timesteps.

Makespan is preferred over sum-of-costs (total individual travel times) because DeepXube uses a single scalar cost per transition, not per-robot costs. It also naturally incentivizes parallel coordination: the heuristic must learn that moving robots simultaneously is cheaper than sequentially.

4 Transition Dynamics

Applying a joint action $(d_1, d_2, \dots, d_K)$ proceeds as follows.

1. For each robot i , compute its target cell (r'_i, c'_i) by adding the direction offset d_i to its current position. If $d_i = \text{WAIT}$, the target is the current cell.
2. If any robot’s target is out of bounds or an obstacle, clamp that robot to its current cell.
3. Check for vertex conflicts (any two robots targeting the same cell) and edge conflicts (any two robots swapping cells). If conflicts exist, the action is invalid and the state is unchanged.
4. Otherwise, update all robot positions simultaneously.

4.1 Non-Decomposability

If each robot were routed independently, their paths would ignore mutual blocking. Robot A may need to move through a cell currently occupied by Robot B, which itself cannot move because Robot C blocks its only escape. These coupled constraints create the combinatorial hardness of MAPF. The heuristic must learn to estimate cost-to-go for the joint configuration, not the sum of individual Manhattan distances, because detour costs from mutual avoidance dominate in congested layouts. The simultaneous formulation makes this coupling explicit: every timestep is a joint decision where one robot’s move constrains what the others can do.

5 Solved Check

For every robot ID $i \in \{1, \dots, K\}$, the cell containing i in the state grid must match the cell containing i in the goal grid. With the grid encoding, a direct array comparison suffices as long as the obstacle mask is excluded (or identically present in both).

6 Visualization

The visualization renders a top-down grid as a high-resolution image sequence, composable into an animated GIF.

6.1 Grid and Background

Each cell is 64×64 pixels. Empty floor cells are filled with `Black_10` (`#ECECEC`). Obstacle cells are filled with black with a subtle 1px `Black_90` inset border to give a hard-edged shelf appearance. Grid lines are 1px `Black_30`. No rounded corners anywhere.

6.2 Robots

Each robot is drawn as a filled circle (radius 24px, centered in its cell) in a distinct brand color.

Robot	Color	Hex
1	Garnet	#73000A
2	Atlantic	#466A9F
3	Congaree	#1F414D
4	Horseshoe	#65780B
5	Rose	#CC2E40
6	Honeycomb	#A49137

A small white triangle inside each circle indicates the direction of the robot’s last move, giving a sense of heading. The robot ID number is rendered in white, bold, centered in the circle.

6.3 Destinations

Each robot’s goal cell is drawn as a hollow square ring (3px stroke) in the matching color, with a faint color wash (15% opacity fill) so the target is visible even when empty. When a robot reaches its destination, the ring pulses briefly (scale up 10% over 3 frames, then back) to signal arrival.

6.4 Trails

The last 4 positions of each robot are shown as fading footprints: small filled circles at 60%, 40%, 20%, and 10% opacity of the robot’s color. This produces a comet-tail effect that makes the coordination dance legible in a GIF.

6.5 Animation

Each action step is one frame, rendered at 4 fps (250ms per frame). Smooth interpolation between cells over 4 sub-frames (16 fps effective) makes motion fluid rather than teleporting. The GIF loops with a 1-second pause at the solved frame.

7 Training Considerations

7.1 Goal-State Pair Sampling

Sample a random valid configuration by placing K robots on distinct non-obstacle cells uniformly at random. The goal is a second independent random placement of K robots on distinct non-obstacle cells. This produces a training pair (state, goal). For curriculum learning, filter pairs by Manhattan distance sum to start with easy instances.

7.2 Reverse Random Walk

From a solved configuration (robots at goal positions), apply N random valid actions. Since all actions are deterministic and reversible (moving robot i left is reversed by moving it right, wait is its own reverse), forward random walks from the goal produce states at known depth N . Start with $N \in [1, 30]$ and anneal up to $[1, 100]$ as training progresses.

7.3 State Space Size

For an 8×8 grid with 10 obstacles (54 free cells) and $K = 4$ robots, the number of distinct states is $P(54, 4) = 54 \times 53 \times 52 \times 51 \approx 7.6$ million. For $K = 6$ on 12×12 with 25 obstacles (119 free cells), $P(119, 6) \approx 2.8 \times 10^{12}$. This is far too large for tabular methods, justifying the learned heuristic.

8 Challenges and Tradeoffs

Action space scaling. With $5K$ actions per step, the A* branching factor grows linearly in robot count. Pruning invalid actions helps (typically only 3–8 actions are valid per state), but search depth also grows. For $K > 6$, consider a simultaneous-move formulation with a learned action-pruning policy.

Deadlocks. Certain configurations produce circular wait patterns (A blocks B blocks C blocks A). The heuristic must learn to assign high cost-to-go to near-deadlock states. Training data from reverse walks naturally avoids deadlocks (since walks start from solved states), which could bias the heuristic. Mitigate by mixing in forward-scrambled states.

Reverse walk quality. Short reverse walks produce states near the goal that are easy to solve. Long reverse walks on congested grids tend to produce states where robots are clustered together, which may not represent the true distribution of hard instances. Use a mix of walk lengths and occasionally restart robots to random positions to improve coverage.

Obstacle layout generalization. If the network is trained on a single obstacle layout, it will not generalize to new warehouses. Train across a distribution of procedurally generated warehouse maps (parameterized by shelf density, aisle width, number of cross-aisles) and include the obstacle mask as part of the input (extend input to $(3, H, W)$ with a binary obstacle channel).